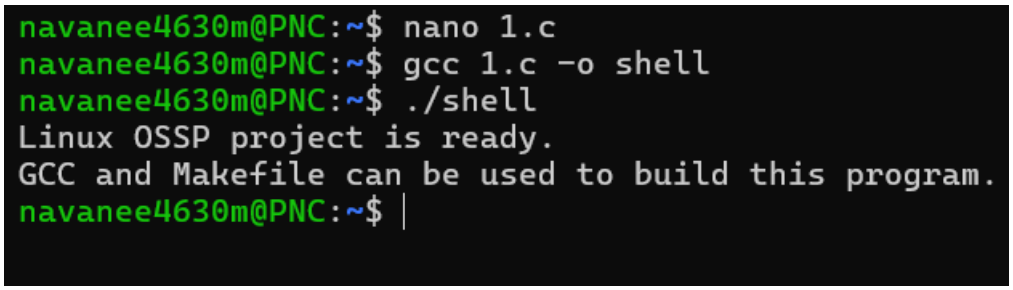
OSSP – SKILL CONTENT DELIVERY CONCEPTS
25CS2104E

Simple C Programs – Copy/Paste Ready Linux Programs

QUESTION 1

To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

```
#include <stdio.h>
int main() {
    printf("Linux OSSP project is ready.\n");
    printf("GCC and Makefile can be used to build this program.\n");
    return 0;
}
```



QUESTION 2

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myshell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        printf("You entered: %s\n", input);
    }

    printf("Shell exited.\n");
    return 0;
}
```

```
navanee4630m@PNC:~$ nano 2.c
navanee4630m@PNC:~$ gcc 2.c -o shell
navanee4630m@PNC:~$ ./shell
myshell>
You entered:
myshell>
You entered:
myshell>
You entered:
myshell> jabn
You entered: jabn
```

QUESTION 3

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5

int main() {
    char history[MAX_HISTORY][100];
    int count = 0;
    char command[100];

    printf("Enter commands (type history to display, exit to stop).\n");
```

```
    while (1) {
        printf("> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            for (int i = 0; i < count; i++)
                printf("%d  %s\n", i + 1, history[i]);
            continue;
        }

        if (count < MAX_HISTORY)
            strcpy(history[count++], command);
    }

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 3.c
navanee4630m@PNC:~$ gcc 3.c -o shell
navanee4630m@PNC:~$ ./shell
Enter commands (type history to display, exit to stop).
> ls
> exit
```

QUESTION 4

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char *token;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    token = strtok(input, " \t\n");

    while (token != NULL) {
        printf("Token: %s\n", token);
        token = strtok(NULL, " \t\n");
    }

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 4.c
navanee4630m@PNC:~$ gcc 4.c -o shell
navanee4630m@PNC:~$ ./shell
Enter command: ls
Token: ls
```

QUESTION 5

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
#include <stdio.h>

int main() {
    char input[] = "'hello $USER world'";
    printf("Single quoted string: %s\n", input);
    printf("Content inside single quotes is treated literally.\n");
    return 0;
}
```

```
navanee4630m@PNC:~$ nano 5.c
navanee4630m@PNC:~$ gcc 5.c -o shell
navanee4630m@PNC:~$ ./shell
Single quoted string: 'hello $USER world'
Content inside single quotes is treated literally.
```

QUESTION 6

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char input[] = "hello\\ world";
    char output[100];
    int j = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\' && input[i + 1] != '\0')
            i++;
        output[j++] = input[i];
    }

    output[j] = '\0';
    printf("Parsed: %s\n", output);

    return 0;
}
```

```
Saranya@saisaranya~$ nano 6.c
Saranya@saisaranya~$ gcc 6.c -o shell
Saranya@saisaranya~$ ./shell
Parsed: hello world
Saranya@saisaranya~$
```

QUESTION 7

Process Synchronization and Child Process Monitoring Using waitpid().

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(2);
        printf("Child finished.\n");
        return 0;
    }

    printf("Parent waiting using waitpid()...\n");
    waitpid(pid, NULL, 0);
    printf("Child has terminated.\n");

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 7.c
navanee4630m@PNC:~$ gcc 7.c -o shell
navanee4630m@PNC:~$ ./shell
Parent waiting using waitpid()...
Child finished.
Child has terminated.
```

QUESTION 8

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char name[50];
    char *value;

    printf("Enter variable name: ");
    scanf("%49s", name);

    value = getenv(name);

    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("Variable is undefined.\n");

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 8.c
navanee4630m@PNC:~$ gcc 8.c -o shell
navanee4630m@PNC:~$ ./shell
Enter variable name: two
Variable is undefined.
```

QUESTION 9

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

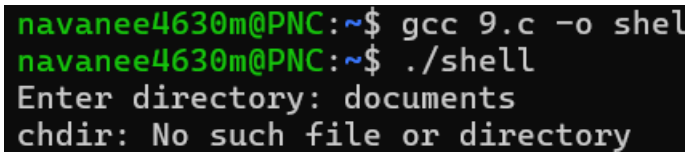
int main() {
    char path[PATH_MAX];

    printf("Enter directory: ");
    scanf("%s", path);

    if (chdir(path) == -1) {
        perror("chdir");
        return 1;
    }

    if (getcwd(path, sizeof(path)) != NULL)
        printf("Current directory: %s\n", path);

    return 0;
}
```



QUESTION 10

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd");
        return 1;
    }

    printf("Current directory: %s\n", cwd);
    printf("Type 'exit' in a shell to process an exit request.\n");

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 10.c
navanee4630m@PNC:~$ gcc 10.c -o shell
navanee4630m@PNC:~$ ./shell
Current directory: /home/navanee4630m
Type 'exit' in a shell to process an exit request.
```

QUESTION 11

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```
#include <stdio.h>
#include <string.h>

#define MAX 5

int main() {
    char history[MAX][100];
    int count = 0;
    char cmd[100];

    for (int i = 0; i < MAX; i++) {
        printf("Command %d: ", i + 1);
        fgets(cmd, sizeof(cmd), stdin);
        cmd[strcspn(cmd, "\n")] = '\0';
        strcpy(history[count++], cmd);
    }

    printf("\nHistory:\n");
    for (int i = 0; i < count; i++)
        printf("%d: %s\n", i + 1, history[i]);

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 111.c
navanee4630m@PNC:~$ gcc 111.c -o shell
navanee4630m@PNC:~$ ./shell
Command 1: ls
Command 2: ls
Command 3: ls
Command 4: exit
```

QUESTION 13

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("input.txt");
        return 1;
    }

    dup2(fd, STDIN_FILENO);
    close(fd);

    char buffer[100];
    if (fgets(buffer, sizeof(buffer), stdin))
        printf("Read from redirected input: %s", buffer);

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 133.c
navanee4630m@PNC:~$ gcc 133.c -o shell
navanee4630m@PNC:~$ ./shell
Read from redirected input: Hello World
```

QUESTION 14

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd < 0) {
        perror("output.txt");
        return 1;
```

```
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("This line is appended.\n");
    return 0;
}
```

```
navanee4630m@PNC:~$ nano 14.c
navanee4630m@PNC:~$ gcc 14.c -o shell
navanee4630m@PNC:~$ ./shell
```

QUESTION 15

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("combined.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
        return 1;

    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);
    close(fd);

    printf("Standard output message.\n");
    fprintf(stderr, "Standard error message.\n");

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 15.c
navanee4630m@PNC:~$ gcc 15.c -o shell
navanee4630m@PNC:~$ ./shell
```

QUESTION 16

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Background job completed.\n");
        return 0;
    }

    printf("Background job started. PID=%d\n", pid);
```

```
    printf("Parent returns to prompt immediately.\n");

    waitpid(pid, NULL, 0);
    return 0;
}
```

```
navanee4630m@PNC:~$ nano 16.c
navanee4630m@PNC:~$ gcc 16.c -o shell
navanee4630m@PNC:~$ ./shell
Background job started. PID=1560
Parent returns to prompt immediately.
Background job completed.
```

QUESTION 17

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(3);
        return 0;
    }

    printf("[1] PID=%d Status=Running\n", pid);
    waitpid(pid, NULL, 0);
    printf("[1] PID=%d Status=Completed\n", pid);

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 17.c
navanee4630m@PNC:~$ gcc 17.c -o shell
navanee4630m@PNC:~$ ./shell
[1] PID=1601 Status=Running
[1] PID=1601 Status=Completed
```

QUESTION 18

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Received signal: %d\n", sig);
}
```

```
int main() {
    signal(SIGUSR1, handler);

    printf("PID: %d\n", getpid());
    printf("Send SIGUSR1 using: kill -USR1 PID\n");

    while (1)
        pause();

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 18.c
navanee4630m@PNC:~$ gcc 18.c -o shell
navanee4630m@PNC:~$ ./shell
PID: 1643
Send SIGUSR1 using: kill -USR1 PID
```

QUESTION 19

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("\nSignal %d received. Program is stable.\n", sig);
}

int main() {
    struct sigaction sa;

    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    printf("PID: %d\n", getpid());

    while (1)
        pause();

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 19.c
navanee4630m@PNC:~$ gcc 19.c -o shell
navanee4630m@PNC:~$ ./shell
PID: 1719
```

QUESTION 20

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("\nSIGTSTP received. Process will stop.\n");
    signal(SIGTSTP, SIG_DFL);
    raise(SIGTSTP);
}

int main() {
    signal(SIGTSTP, handler);

    printf("PID: %d\n", getpid());
    printf("Press Ctrl+Z to suspend.\n");

    while (1) {
        printf("Running...\n");
        sleep(1);
    }

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 20.c
navanee4630m@PNC:~$ gcc 20.c -o shell
navanee4630m@PNC:~$ ./shell
PID: 1893
Press Ctrl+Z to suspend.
Running...
Running...
Running...
^Z
SIGTSTP received. Process will stop.

[1]+  Stopped                 ./shell
```

QUESTION 21

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int a = 10, b = 20;

    printf("Module 1 result: %d\n", add(a, b));
    printf("End-to-end module test completed.\n");
```

```
    return 0;
}
```

```
navanee4630m@PNC:~$ nano 221.c
navanee4630m@PNC:~$ gcc 221.c -o shell
navanee4630m@PNC:~$ ./shell
Module 1 result: 30
End-to-end module test completed.
```

QUESTION 22

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(10 * sizeof(int));

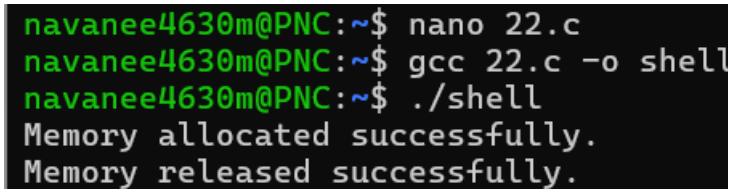
    if (ptr == NULL)
        return 1;

    for (int i = 0; i < 10; i++)
        ptr[i] = i;

    printf("Memory allocated successfully.\n");

    free(ptr);
    printf("Memory released successfully.\n");

    return 0;
}
```



QUESTION 23

To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    clock_t start = clock();

    for (int i = 0; i < 3; i++) {
        if (fork() == 0) {
            printf("Pipeline job %d, PID=%d\n", i + 1, getpid());
            sleep(1);
            return 0;
        }
    }

    for (int i = 0; i < 3; i++)
        wait(NULL);

    clock_t end = clock();

    printf("All jobs completed.\n");
    printf("CPU time: %.3f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);

    return 0;
}
```

```
navanee4630m@PNC:~$ nano 23.c
navanee4630m@PNC:~$ gcc 23.c -o shell
navanee4630m@PNC:~$ ./shell
Pipeline job 1, PID=2036
Pipeline job 2, PID=2037
Pipeline job 3, PID=2038
All jobs completed.
CPU time: 0.000 seconds
```

QUESTION 24

To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

```
#include <stdio.h>

int main() {
    printf("Project structure check\n");
    printf("src/       - source files\n");
    printf("include/   - header files\n");
    printf("build/     - compiled files\n");
    printf("README.md  - project documentation\n");
    printf("Makefile   - build instructions\n");
    return 0;
}
```

```
navanee4630m@PNC:~$ nano 24.c
navanee4630m@PNC:~$ gcc 24.c -o shell
navanee4630m@PNC:~$ ./shell
Project structure check
src/       - source files
include/   - header files
build/     - compiled files
README.md  - project documentation
Makefile   - build instructions
```

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Demo signal received: %d\n", sig);
}

int main() {
    signal(SIGUSR1, handler);

    printf("OSSP Demo\n");
    printf("PID: %d\n", getpid());
    printf("Feature 1: Process execution\n");
    printf("Feature 2: Pipelines\n");
    printf("Feature 3: Signal handling\n");

    printf("Sending demonstration signal...\n");
    raise(SIGUSR1);

    printf("Final project review completed.\n");
    return 0;
}
```

```
navanee4630m@PNC:~$ nano 225.c
navanee4630m@PNC:~$ gcc 225.c -o shell
navanee4630m@PNC:~$ ./shell
OSSP Demo
PID: 2124
Feature 1: Process execution
Feature 2: Pipelines
Feature 3: Signal handling
Sending demonstration signal...
Demo signal received: 10
Final project review completed.
```